Graph Algorithms

Assoc.Prof. Suphakant Phimoltares, Ph.D.
Advanced Virtual and Intelligent Computing (AVIC) Center

Agenda

- Graph representations/Breadth-first search/Depth-first search/Topological sorting/Strongly connected components
- Minimum spanning tree: Kruskal's algorithm/Prim's algorithm
- Single-source shortest paths: Bellman-Ford algorithm/Single-source shortest paths in DAG/Dijkstra's algorithm
- All-pairs shortest paths: Shortest paths and matrix multiplication/Floyd-Warshall algorithm

Graph representations

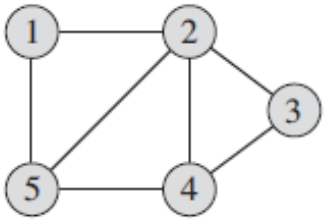
Adjacency list representation

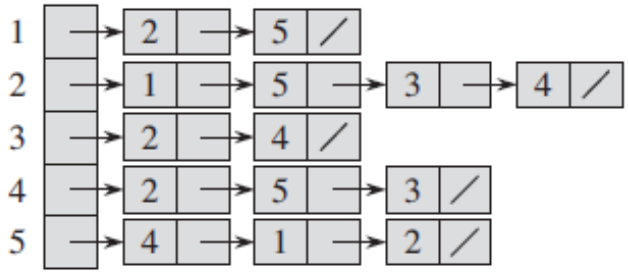
- a compact way to represent ***sparse*** graphs—those for which $|E|$ is much less than $|V|^2$.
- For each $u \in V$, the adjacency list $Adj[u]$contains all the vertices v such that there is an edge $(u, v) \in E$.
- For a weighted graph, weight $w(u, v)$ of the edge $(u, v) \in E$ with vertex v in u's adjacency list is stored.

Graph representations

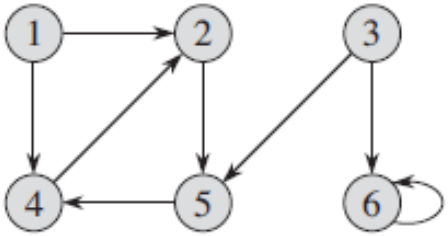
Adjacency matrix representation

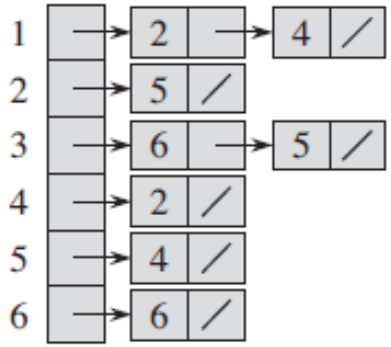
- Suitable for ***dense*** graphs—$|E|$ is close to $|V|^2$.
- Size of matrix is $|V| \times |V|$.
- $A = a(i,j)$ where $a(i,j) = \begin{cases} 1 \text{ if } (i,j) \in E, \\ 0 \quad \text{otherwise} \end{cases}$
- For a weighted graph, weight $w(u,v)$ of the edge $(u,v) \in E$ is stored as the entry in row u and column v of the adjacency matrix.





	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0





	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

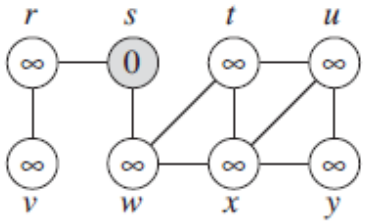
Breadth-first search

- To search graph
- All vertices start out white and may later become gray and then black.
- Breadth-first search constructs a breadth-first tree, initially containing only its root, which is the source vertex s.
- $u.color$: color of vertex u, $u.\pi$: predecessor of vertex u, $u.d$ holds the distance from the source s to vertex u.

```
BFS(G, s)
 1  for each vertex u ∈ G.V − {s}
 2       u.color = WHITE
 3       u.d = ∞
 4       u.π = NIL
 5  s.color = GRAY
 6  s.d = 0
 7  s.π = NIL
 8  Q = ∅
 9  ENQUEUE(Q, s)
10  while Q ≠ ∅
11       u = DEQUEUE(Q)
12       for each v ∈ G.Adj[u]
13            if v.color == WHITE
14                 v.color = GRAY
15                 v.d = u.d + 1
16                 v.π = u
17                 ENQUEUE(Q, v)
18       u.color = BLACK
```

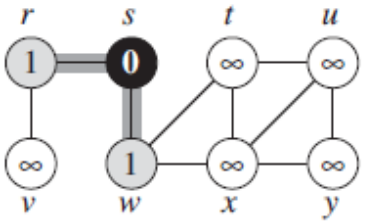
Running time ?

(a)



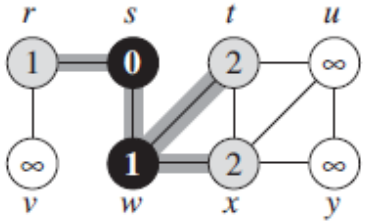
Q | s | | |
0

(b)



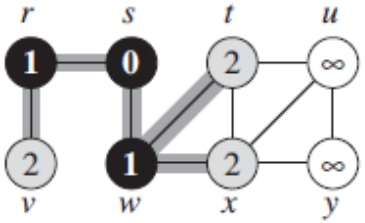
Q | w | r | |
1 1

(c)



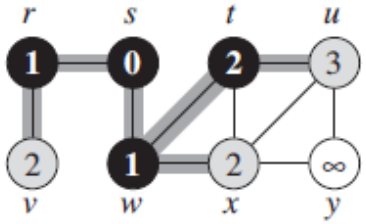
Q | r | t | x |
1 2 2

(d)



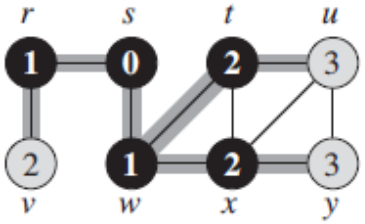
Q | t | x | v |
2 2 2

(e)



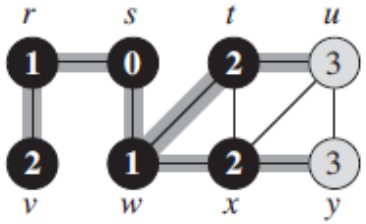
Q | x | v | u |
2 2 3

(f)



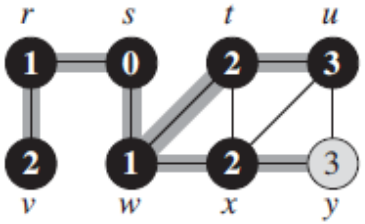
Q | v | u | y |
2 3 3

(g)



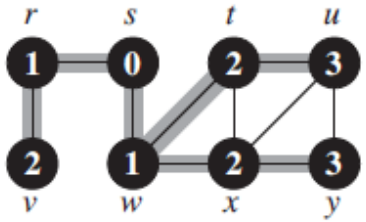
Q | u | y | |
3 3

(h)



Q | y | | |
3

(i)



Q $\emptyset$

Breadth-first search

- A breadth-first tree $G_\pi = (V_\pi, E_\pi)$ where
  $V_\pi = \{v \in V : v.\pi \neq \text{NIL}\} \cup \{s\}$
  $E_\pi = \{(v.\pi, v) : v \in V_\pi - \{s\}\}$
- Edges in E_π are called ***tree edges***.

Depth-first search

- To search “deeper” in the graph whenever possible.
- If any undiscovered vertices remain, then depth-first search selects one of them as a new source.
- Unlike breadth-first search, whose predecessor subgraph forms a tree, the predecessor subgraph produced by a depth-first search may be composed of several trees, because the search may repeat from multiple sources.

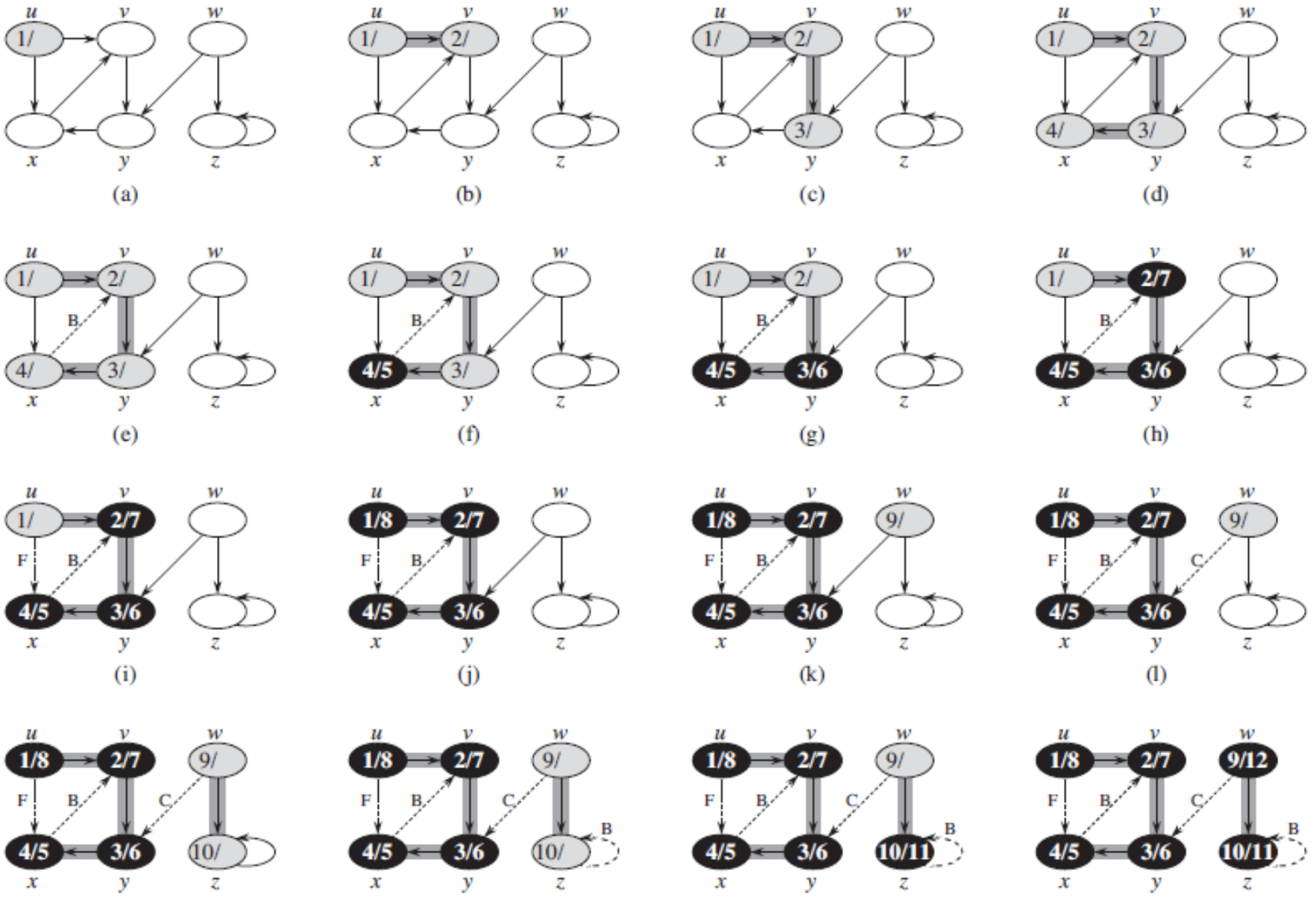
Depth-first search

- A ***depth-first forest*** of a depth-first search $G_\pi = (V, E_\pi)$ where
  $$E_\pi = \{(v.\pi, v) : v \in V \text{ and } v.\pi \neq \text{NIL}\}$$
- Edges in E_π are called ***tree edges***.
- Each vertex has two timestamps: the first timestamp $v.d$ records when v is first discovered (and grayed), and the second timestamp $v.f$ records when the search finishes examining v's adjacency list (and blackens v).

```
DFS(G)
1  for each vertex u ∈ G.V
2      u.color = WHITE
3      u.π = NIL
4  time = 0
5  for each vertex u ∈ G.V
6      if u.color == WHITE
7          DFS-VISIT(G, u)
```

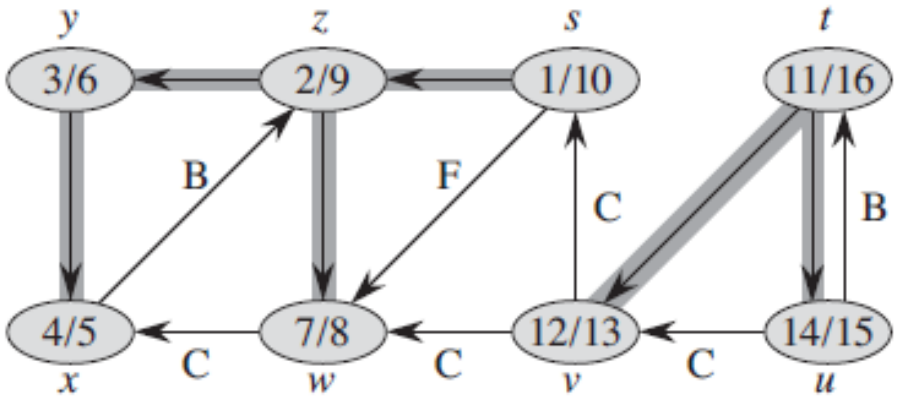
```
DFS-VISIT(G, u)
 1  time = time + 1                  // white vertex u has just been discovered
 2  u.d = time
 3  u.color = GRAY
 4  for each v ∈ G.Adj[u]            // explore edge (u, v)
 5      if v.color == WHITE
 6          v.π = u
 7          DFS-VISIT(G, v)
 8  u.color = BLACK                  // blacken u; it is finished
 9  time = time + 1
10  u.f = time
```

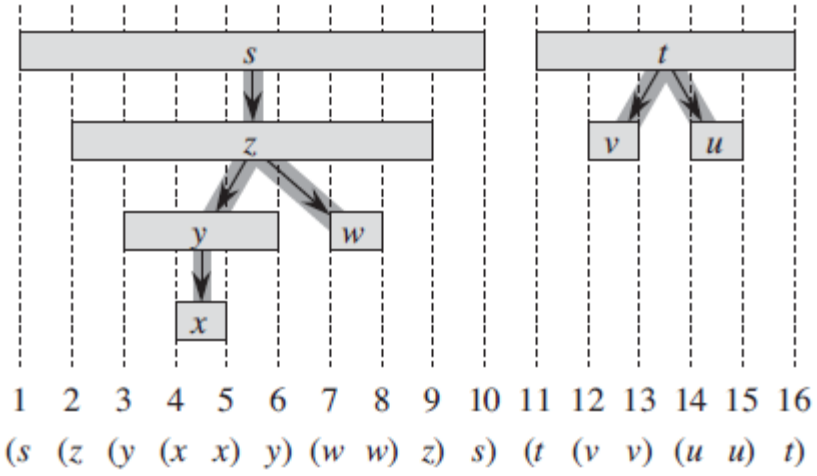
Running time ?

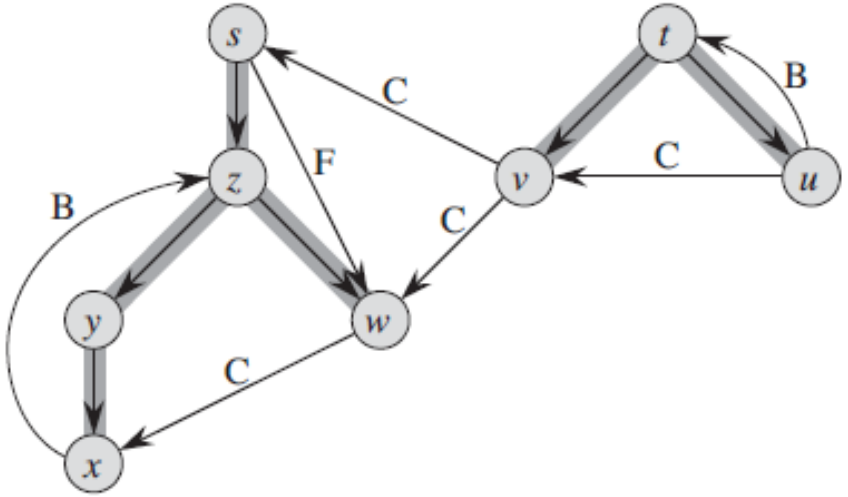

u v w
1/
x y z
(a)
u v w
1/ 2/
x y z
(b)
u v w
1/ 2/
3/
x y z
(c)
u v w
1/ 2/
4/ 3/
x y z
(d)
u v w
1/ 2/
B
4/ 3/
x y z
(e)
u v w
1/ 2/
B
4/5 3/
x y z
(f)
u v w
1/ 2/
B
4/5 3/6
x y z
(g)
u v w
1/ 2/7
B
4/5 3/6
x y z
(h)
u v w
1/ 2/7
F B
4/5 3/6
x y z
(i)
u v w
1/8 2/7
F B
4/5 3/6
x y z
(j)
u v w
1/8 2/7 9/
F B
4/5 3/6
x y z
(k)
u v w
1/8 2/7 9/
F B C
4/5 3/6
x y z
(l)
u v w
1/8 2/7 9/
F B C
4/5 3/6 10/
x y z
u v w
1/8 2/7 9/
F B C
4/5 3/6 10/ B
x y z
u v w
1/8 2/7 9/
F B C
4/5 3/6 10/11 B
x y z
u v w
1/8 2/7 9/12
F B C
4/5 3/6 10/11 B
x y z

Depth-first search

- DFS contains ***parenthesis structure***.
- Left parenthesis (u represents discovery of vertex u.
- Right parenthesis u) represents finishing of vertex u.


y
z
s
t
3/6
2/9
1/10
11/16
B
F
C
B
4/5
7/8
12/13
14/15
C
C
C
x
w
v
u


s
z
y
w
x
t
v
u
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
(s (z (y (x x) y) (w w) z) s) (t (v v) (u u) t)


s
t
C
B
z
F
v
C
u
B
C
y
w
C
x

Depth-first search

Edge classification

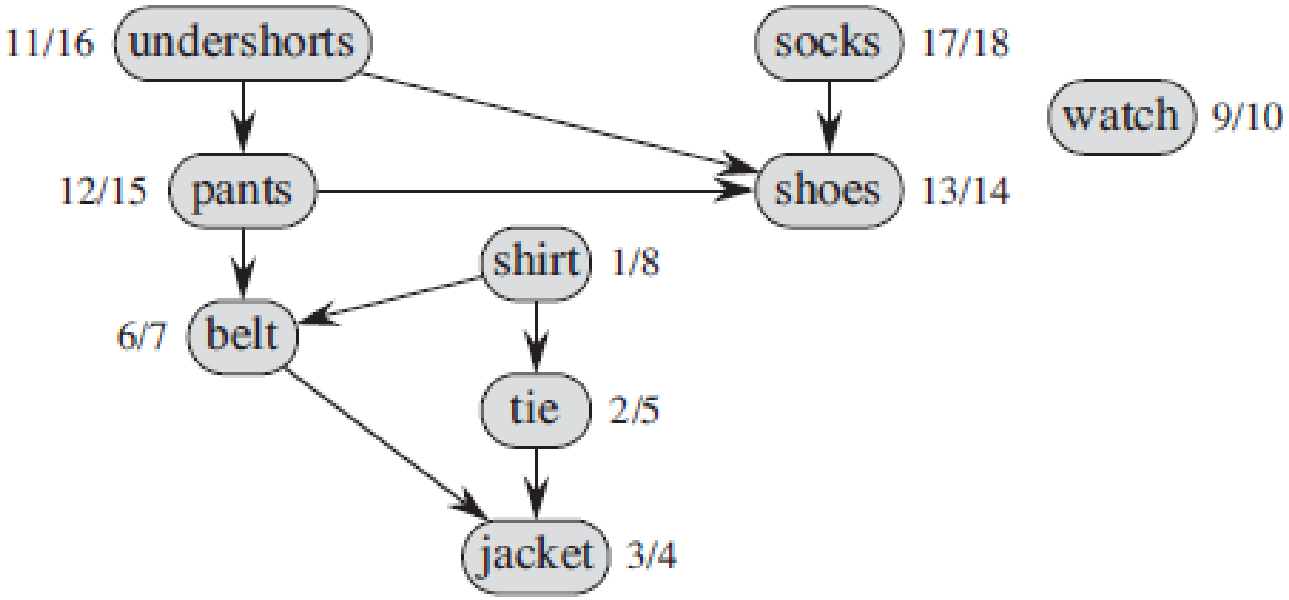
- ***Tree edge.*** Edge (u, v) is a tree edge if v was first discovered by exploring edge (u, v).
- ***Back edge.*** Edge (u, v) connecting a vertex u to an ancestor v in a depth-first tree. This might include self-loops.
- ***Forward edge.*** nontree edges (u, v) connecting a vertex u to a descendant v in a depth-first tree.
- ***Cross edge.*** All other edges.

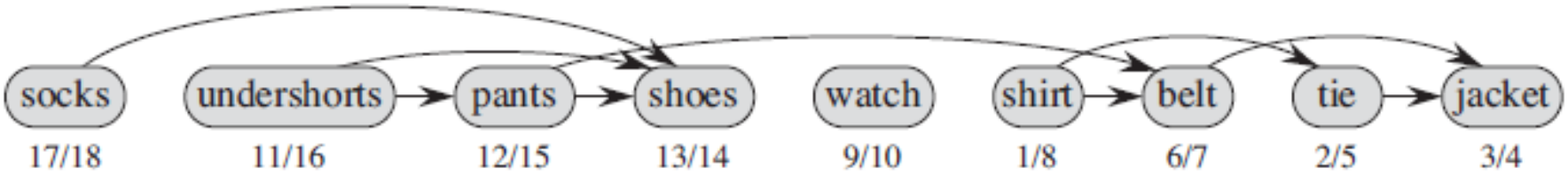
Topological sorting

- Depth-first search can be used for a directed acyclic graph.

```
TOPOLOGICAL-SORT(G)
1  call DFS(G) to compute finishing times v.f for each vertex v
2  as each vertex is finished, insert it onto the front of a linked list
3  return the linked list of vertices
```

Running time ?


11/16
undershorts
socks
17/18
watch
9/10
12/15
pants
shoes
13/14
shirt
1/8
6/7
belt
tie
2/5
jacket
3/4


socks
17/18
undershorts
11/16
pants
12/15
shoes
13/14
watch
9/10
shirt
1/8
belt
6/7
tie
2/5
jacket
3/4

Strongly connected components

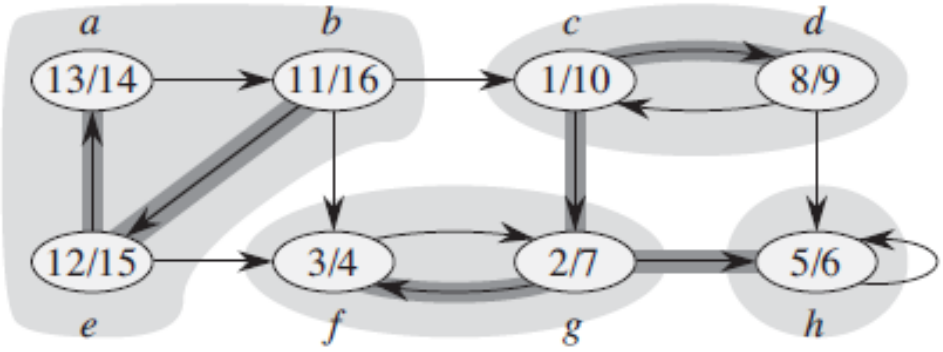
- A strongly connected component of a directed graph $G = (V, E)$ is a set of vertices $C \subseteq V$ such that for every pair of vertices u and v in C, vertices u and v are reachable from each other.
- Define $G^T = (V, E^T)$, where $E^T = \{(u, v) : (v, u) \in E\}$.

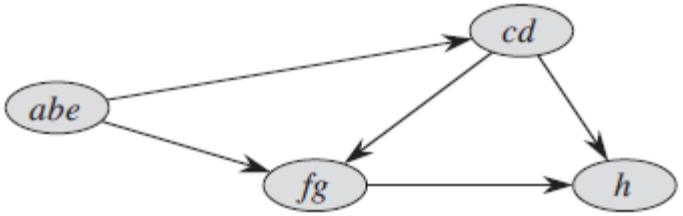
Strongly connected components

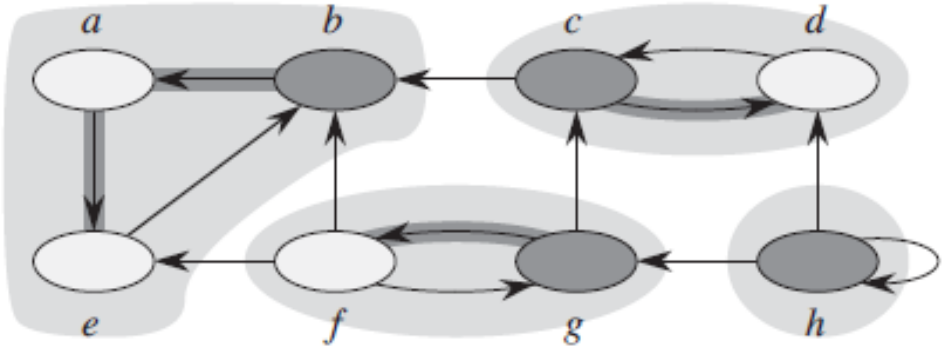
STRONGLY-CONNECTED-COMPONENTS(G)

1 call DFS(G) to compute finishing times $u.f$ for each vertex u
2 compute G^T
3 call DFS(G^T), but in the main loop of DFS, consider the vertices
in order of decreasing $u.f$ (as computed in line 1)
4 output the vertices of each tree in the depth-first forest formed in line 3 as a
separate strongly connected component

Running time ?


a
b
c
d
13/14
11/16
1/10
8/9
12/15
3/4
2/7
5/6
e
f
g
h


cd
abe
fg
h


a
b
c
d
e
f
g
h

Minimum spanning tree

Applications

- Electronic circuit designs.
- Network planning.

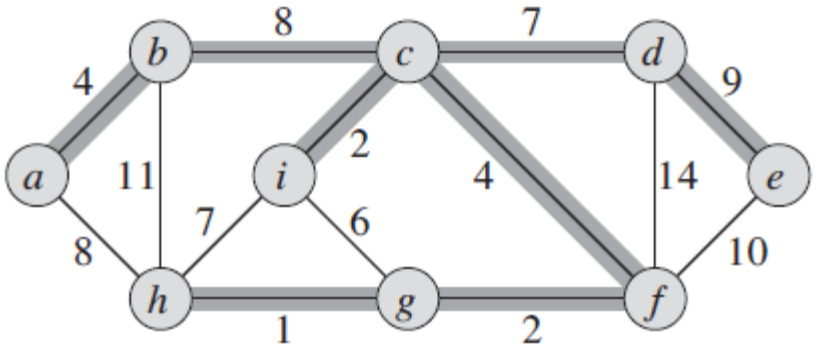
Objective

- To find an acyclic subset $T \subseteq E$ that connects all of the vertices and whose total weight

$$w(T) = \sum_{(u,v)\in T} w(u,v)$$

Minimum spanning tree

- T is called ***spanning tree***.
- T with minimum total weight is ***minimum spanning tree***.
- Use greedy algorithm making one of several possible choices to solve this problem.



Minimum spanning tree

Generic method

```
GENERIC-MST(G, w)
1  A = ∅
2  while A does not form a spanning tree
3      find an edge (u, v) that is safe for A
4      A = A ∪ {(u, v)}
5  return A
```

- Line 3 is very important!
- **Algorithms of Kruskal and Prim**

Minimum spanning tree

Difference between two algorithms

- In Kruskal's algorithm, the set A is a forest whose vertices are all those of the given graph. The safe edge added to A is always a least-weight edge in the graph that connects two distinct components.
- In Prim's algorithm, the set A forms a single tree. The safe edge added to A is always a least-weight edge connecting the tree to a vertex not in the tree.

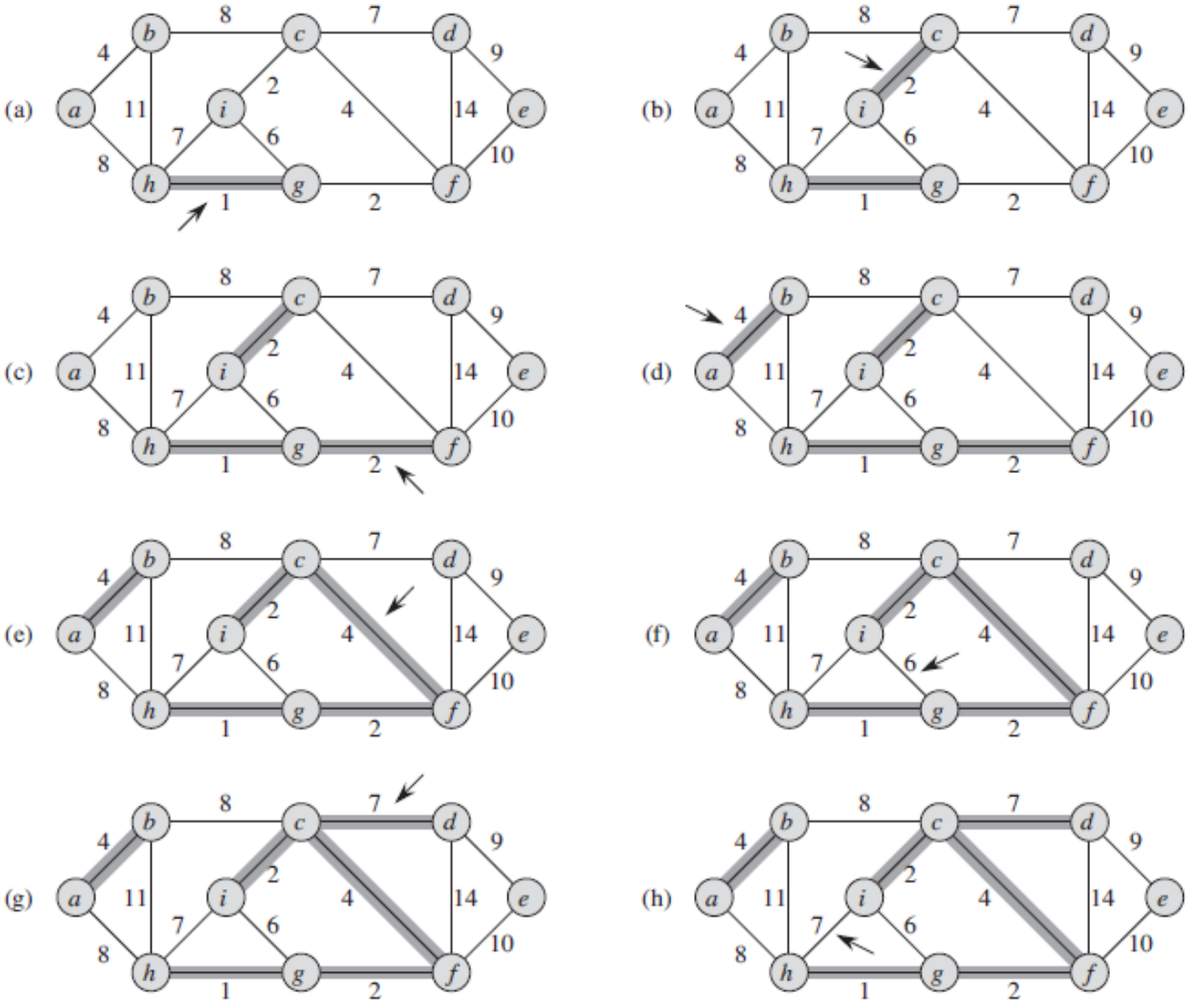
Kruskal's algorithm

- Kruskal's algorithm finds a safe edge to add to the growing forest by finding, of all the edges that connect any two trees in the forest, an edge (u, v) of least weight.
- It qualifies as a greedy algorithm because at each step it adds to the forest an edge of least possible weight.
- It uses a disjoint-set data structure to maintain several disjoint sets of elements.

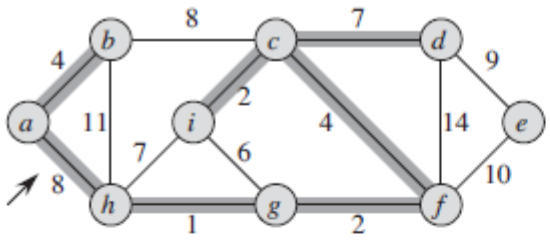
Kruskal's algorithm

```
MST-KRUSKAL(G, w)
1  A = ∅
2  for each vertex v ∈ G.V
3       MAKE-SET(v)
4  sort the edges of G.E into nondecreasing order by weight w
5  for each edge (u, v) ∈ G.E, taken in nondecreasing order by weight
6       if FIND-SET(u) ≠ FIND-SET(v)
7            A = A ∪ {(u, v)}
8            UNION(u, v)
9  return A
```

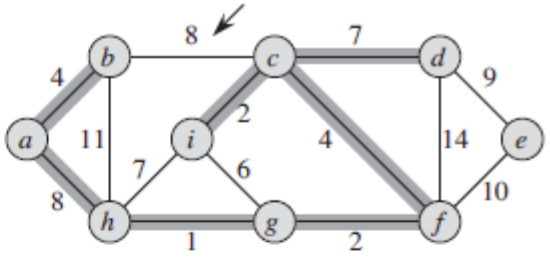
Running time ?


(a)
(b)
(c)
(d)
(e)
(f)
(g)
(h)
a
b
c
d
e
f
g
h
i
4
8
11
8
7
2
4
7
6
14
9
10
1
2

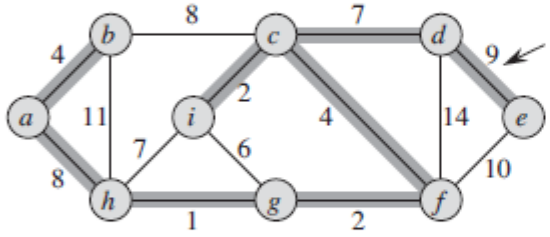
(i)


a
b
c
d
e
f
g
h
i
4
8
8
7
9
11
7
2
4
14
6
10
8
1
2

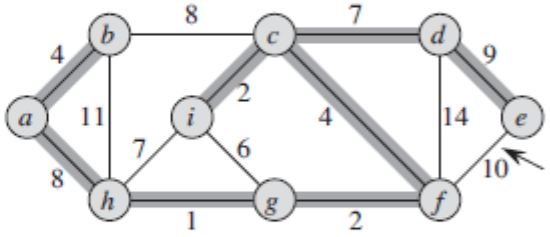
(j)


a
b
c
d
e
f
g
h
i
4
8
8
7
9
11
7
2
4
14
6
10
8
1
2

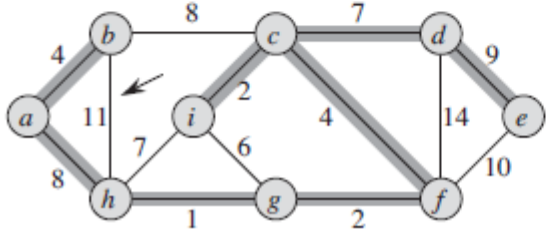
(k)


a
b
c
d
e
f
g
h
i
4
8
8
7
9
11
7
2
4
14
6
10
8
1
2

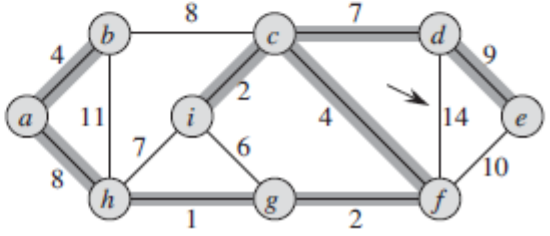
(l)


a
b
c
d
e
f
g
h
i
4
8
8
7
9
11
7
2
4
14
6
10
8
1
2

(m)


a
b
c
d
e
f
g
h
i
4
8
8
7
9
11
7
2
4
14
6
10
8
1
2

(n)


a
b
c
d
e
f
g
h
i
4
8
8
7
9
11
7
2
4
14
6
10
8
1
2

Prim's algorithm

- Each step adds to the tree A a light edge that connects A to an isolated vertex—one on which no edge of A is incident.
- During execution of the algorithm, all vertices that are *not* in the tree reside in a min-priority queue Q based on a key attribute. For each vertex v, the attribute $v.key$ is the minimum weight of any edge connecting to a vertex in the tree; by convention, $v.key = \infty$ if there is no such edge.

Prim's algorithm

- The algorithm implicitly maintains the set A from GENERIC-MST as

$$A = \{(v, v.\pi) : v \in V - \{r\} - Q\}.$$

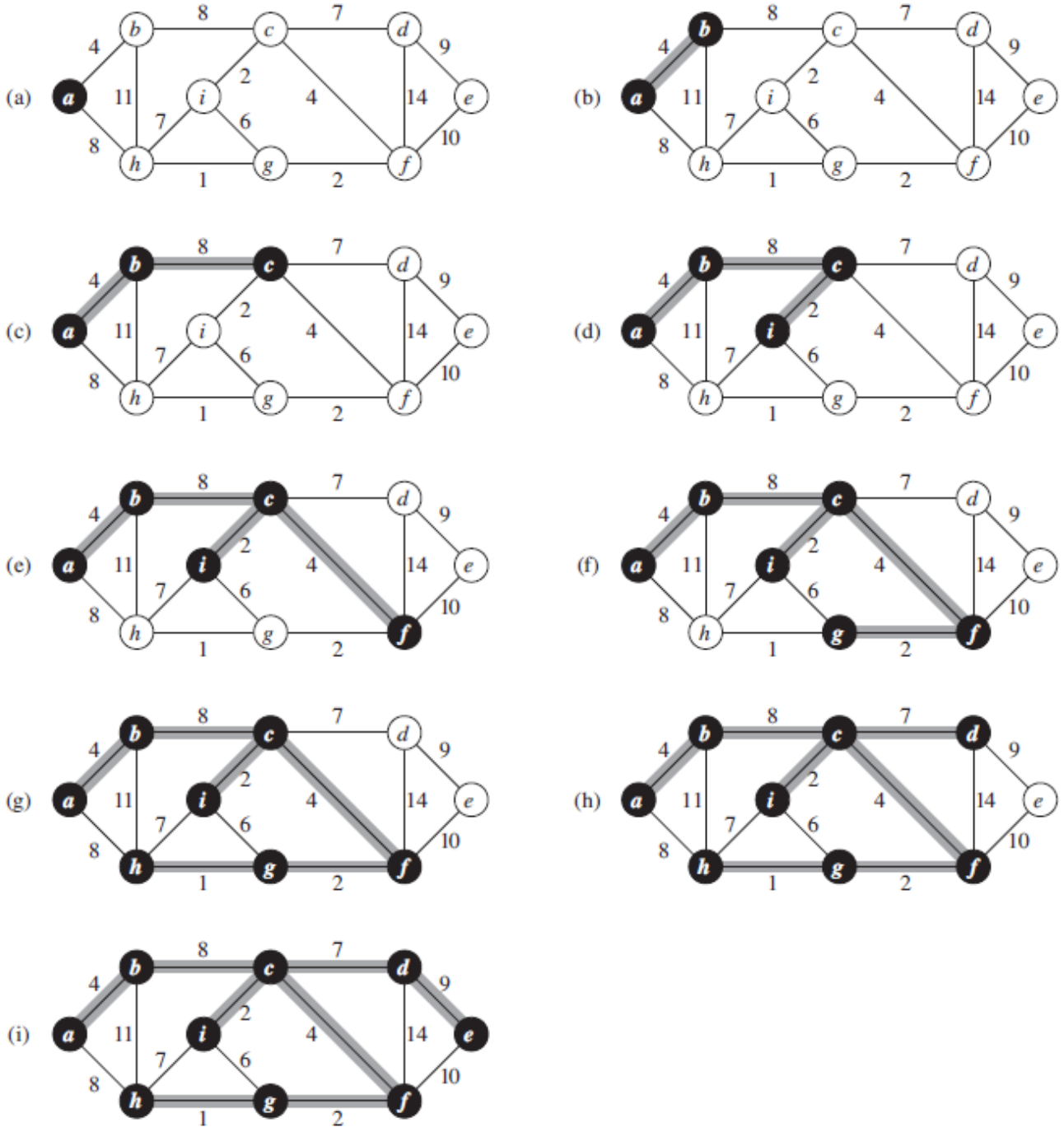
- When the algorithm terminates, the min-priority queue Q is empty; the minimum spanning tree A for G is thus

$$A = \{(v, v.\pi) : v \in V - \{r\}\}.$$

Prim's algorithm

```
MST-PRIM(G, w, r)
 1  for each u ∈ G.V
 2      u.key = ∞
 3      u.π = NIL
 4  r.key = 0
 5  Q = G.V
 6  while Q ≠ ∅
 7      u = EXTRACT-MIN(Q)
 8      for each v ∈ G.Adj[u]
 9          if v ∈ Q and w(u, v) < v.key
10              v.π = u
11              v.key = w(u, v)
```

Running time ?


(a)
(b)
(c)
(d)
(e)
(f)
(g)
(h)
(i)
a
b
c
d
e
f
g
h
i
4
8
11
7
8
2
6
4
7
14
9
10
1
2

Single-Source Shortest Paths

Applications

- Shortest route from a city to the others.

Definition

- For a directed graph, the weight $w(p)$ of path $p = \langle v_0, v_1, \ldots, v_k \rangle$ is the sum of the weights

$$w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$$

Single-Source Shortest Paths

- The ***shortest-path weight*** $\delta(u, v)$ from u to v is defined by

$$\delta(u, v) = \begin{cases} \min\left\{w(p) : u \xrightarrow{p} v\right\} & \text{if there is a path from } u \text{ to } v \\ \infty & \text{otherwise} \end{cases}$$

- A ***shortest path*** from vertex u to vertex v is then defined as any path p with weight $w(p) = \delta(u, v)$.

Single-Source Shortest Paths

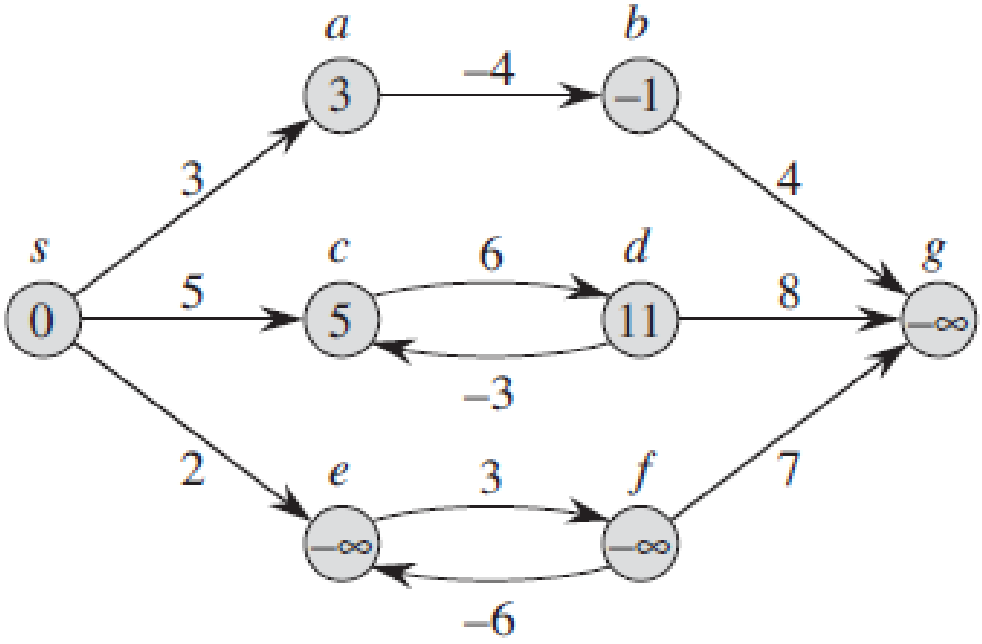
Variants:

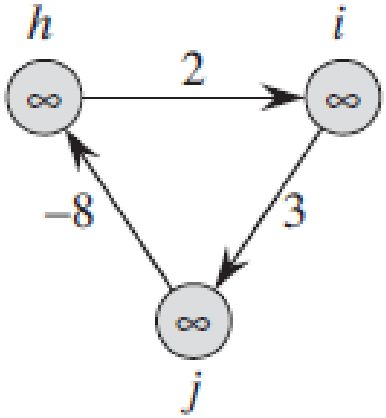
- Single-destination shortest-paths problem.
- Single-pair shortest-path problem.
- All-pairs shortest-paths problem.

Negative-weight edges:

- If the graph contains a negative-weight cycle reachable from s, however, shortest-path weights are not well defined.

Single-Source Shortest Paths





Single-Source Shortest Paths

Representing shortest paths

- Define **predecessor subgraph** $G_\pi = (V_\pi, E_\pi)$ where

$$V_\pi = \{v \in V : v.\pi \neq \text{NIL}\} \cup \{s\}$$

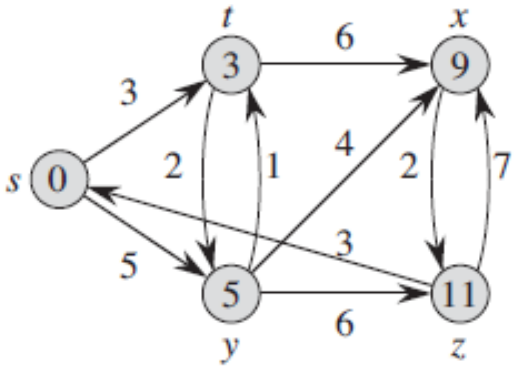
$$E_\pi = \{(v.\pi, v) \in E : v \in V_\pi - \{s\}\}$$

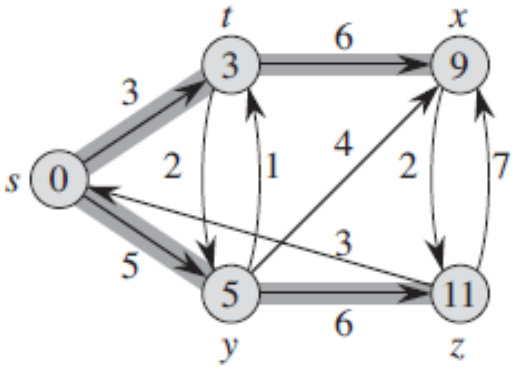
- A shortest-paths tree is like the breadth-first tree, but it contains shortest paths from the source defined in terms of edge weights instead of numbers of edges.

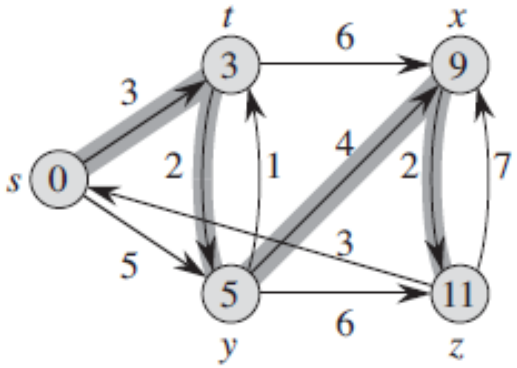
Single-Source Shortest Paths

- A ***shortest-paths tree*** rooted at s is a directed subgraph $G' = (V', E')$ where $V' \subseteq V$ and $E' \subseteq E$, such that
  - V' is the set of vertices reachable from s in G,
  - G' forms a rooted tree with root s, and
  - for all $v \in V'$, the unique simple path from s to v in G' is a shortest path from s to v in G.

Single-Source Shortest Paths



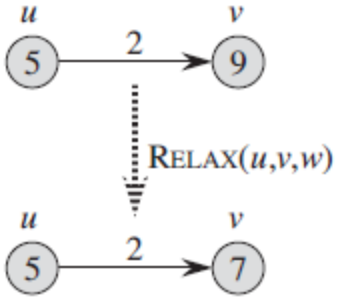


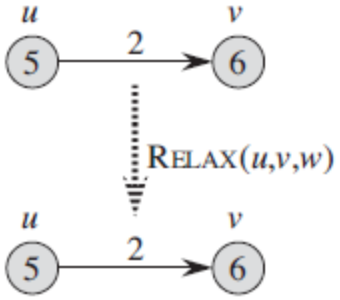


Single-Source Shortest Paths

Initialization and relaxation

```
INITIALIZE-SINGLE-SOURCE(G, s)
1  for each vertex v ∈ G.V
2      v.d = ∞
3      v.π = NIL
4  s.d = 0
```



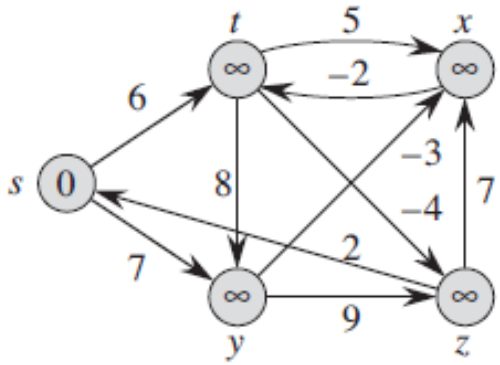



```
RELAX(u, v, w)
1  if v.d > u.d + w(u, v)
2      v.d = u.d + w(u, v)
3      v.π = u
```

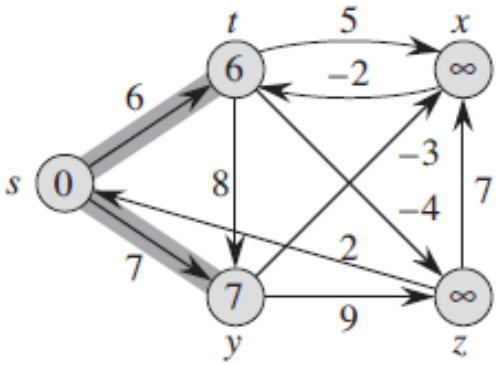
Bellman-Ford algorithm

```
BELLMAN-FORD(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  for i = 1 to |G.V| − 1
3       for each edge (u, v) ∈ G.E
4            RELAX(u, v, w)
5  for each edge (u, v) ∈ G.E
6       if v.d > u.d + w(u, v)
7            return FALSE
8  return TRUE
```

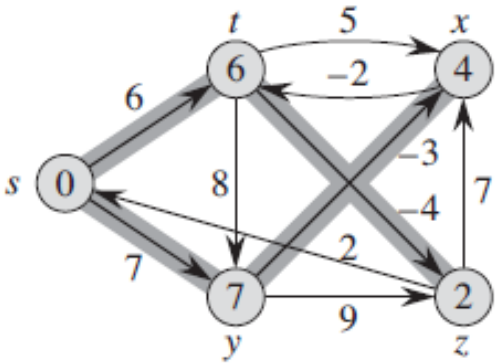
Running time ?


t
5
x
∞
−2
∞
6
−3
s
0
8
−4
7
7
2
∞
∞
9
y
z

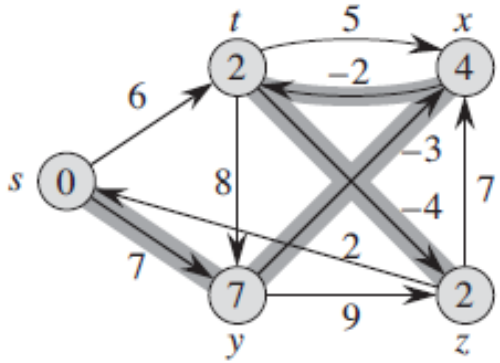
(a)


t
5
x
6
−2
∞
6
−3
s
0
8
−4
7
7
2
7
∞
9
y
z

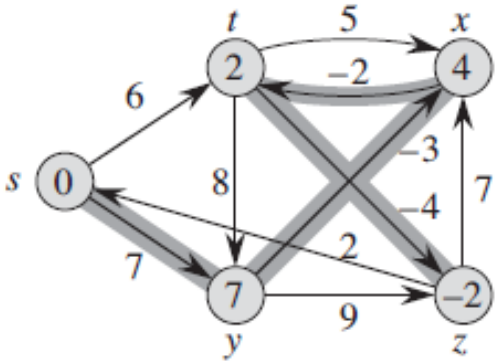
(b)


t
5
x
6
−2
4
6
−3
s
0
8
−4
7
7
2
7
2
9
y
z

(c)


t
5
x
2
−2
4
6
−3
s
0
8
−4
7
7
2
7
2
9
y
z

(d)


t
5
x
2
−2
4
6
−3
s
0
8
−4
7
7
2
7
−2
9
y
z

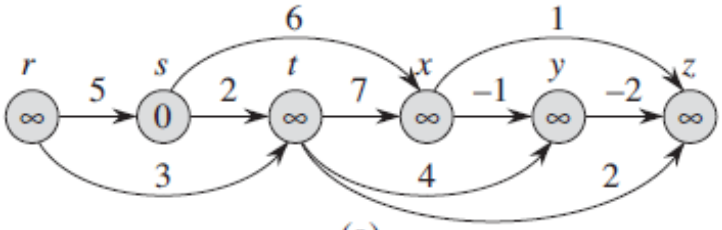
(e)

Single-source shortest paths in DAG

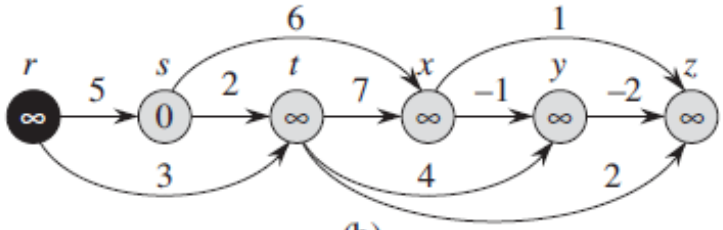
- Based on topological sorting

```
DAG-SHORTEST-PATHS(G, w, s)
1  topologically sort the vertices of G
2  INITIALIZE-SINGLE-SOURCE(G, s)
3  for each vertex u, taken in topologically sorted order
4       for each vertex v ∈ G.Adj[u]
5            RELAX(u, v, w)
```

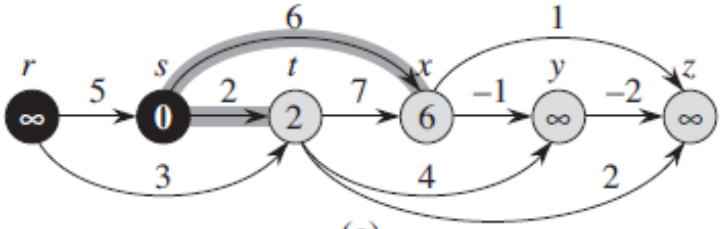
Running time ?


r
s
t
x
y
z
∞
0
∞
∞
∞
∞
5
2
7
−1
−2
6
1
3
4
2

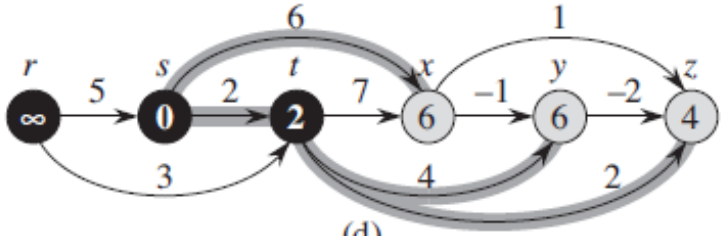
(a)


r
s
t
x
y
z
∞
0
∞
∞
∞
∞
5
2
7
−1
−2
6
1
3
4
2

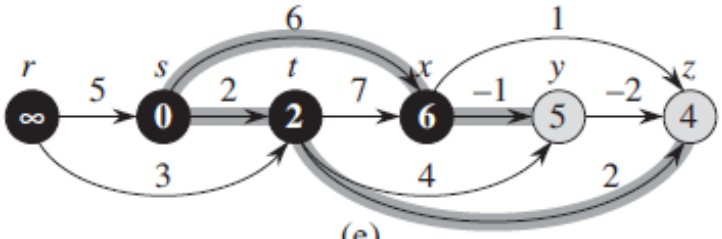
(b)


r
s
t
x
y
z
∞
0
2
6
∞
∞
5
2
7
−1
−2
6
1
3
4
2

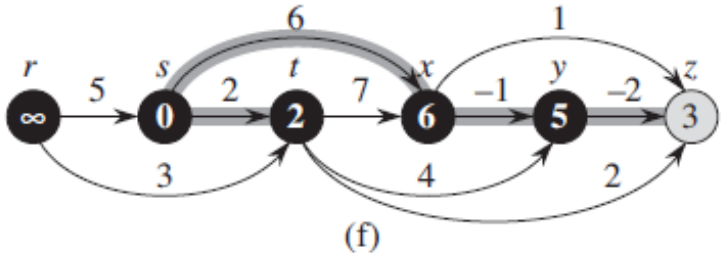
(c)


r
s
t
x
y
z
∞
0
2
6
6
4
5
2
7
−1
−2
6
1
3
4
2

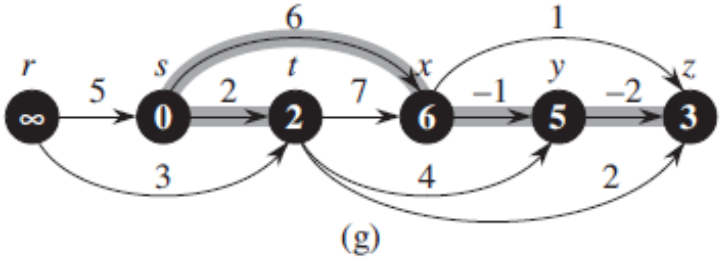
(d)


r
s
t
x
y
z
∞
0
2
6
5
4
5
2
7
−1
−2
6
1
3
4
2

(e)


r
s
t
x
y
z
∞
0
2
6
5
3
5
2
7
−1
−2
6
1
3
4
2

(f)


r
s
t
x
y
z
∞
0
2
6
5
3
5
2
7
−1
−2
6
1
3
4
2

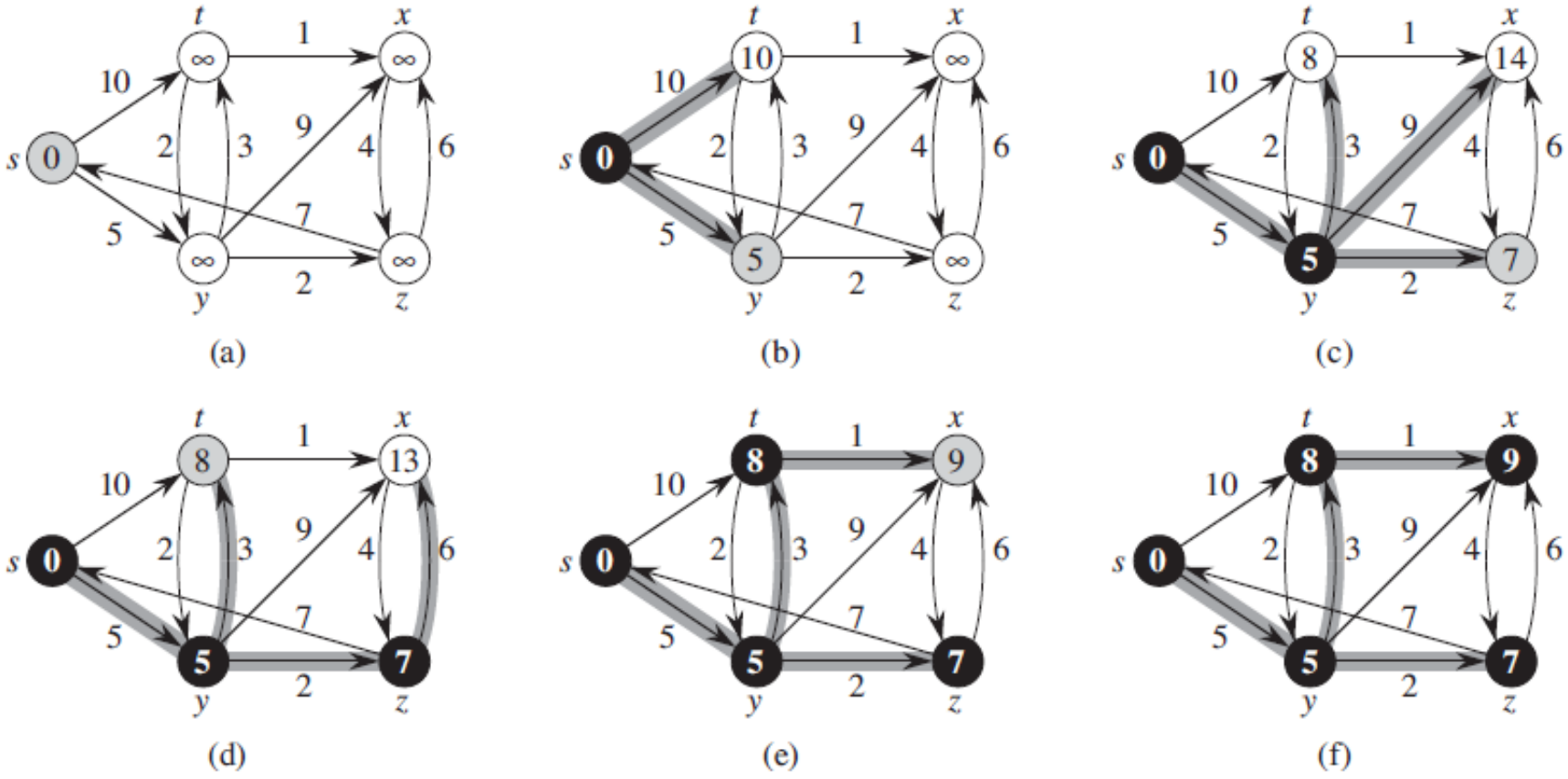
(g)

Dijkstra's algorithm

- It always chooses "closet" vertex in $V - S$ to add to set S.

```
DIJKSTRA(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  S = ∅
3  Q = G.V
4  while Q ≠ ∅
5      u = EXTRACT-MIN(Q)
6      S = S ∪ {u}
7      for each vertex v ∈ G.Adj[u]
8          RELAX(u, v, w)
```

Running time ?


t
x
s
y
z
10
1
2
3
9
4
6
5
7
2
∞
0
10
5
8
14
13
9
(a)
(b)
(c)
(d)
(e)
(f)

All-pairs shortest paths

- If all edge weights are nonnegative, we can use Dijkstra's algorithm. With the linear-array implementation of the min-priority queue, the running time is $O(V^3)$.
- If the graph has negative-weight edges, we must run the slower Bellman-Ford algorithm once from each vertex. The resulting running time is $O(V^2E)$, which on a dense graph is $O(V^4)$.

All-pairs shortest paths

- If vertices are numbered, an $n \times n$ matrix $W = (w_{ij})$ is defined, where w_{ij} is the weight of directed edge (i, j). In case of $i = j$, $w_{ij} = 0$, but if $(i, j) \notin E$, $w_{ij} = \infty$.
- ***Predecessor matrix*** is defined as $\Pi = (\pi_{ij})$, where π_{ij} is NIL if either $i = j$ or there is no path from i to j, and otherwise π_{ij} is the predecessor of j on some shortest path from i.

All-pairs shortest paths

```
PRINT-ALL-PAIRS-SHORTEST-PATH(Π, i, j)
1  if i == j
2      print i
3  elseif π_ij == NIL
4      print "no path from" i "to" j "exists"
5  else PRINT-ALL-PAIRS-SHORTEST-PATH(Π, i, π_ij)
6      print j
```

Shortest paths and matrix multiplication

A recursive solution to the all-pairs shortest-paths problem

- Let $l_{ij}^{(0)} = \begin{cases} 0 \text{ if } i = j \\ \infty \text{ if } i \neq j \end{cases}$.
- The minimum weight of any path from i to j consisting of at most m edges, obtained by looking at all possible predecessors k of j.

$$l_{ij}^{(m)} = \min\left(l_{ij}^{(m-1)}, \min_{1 \le k \le n}\left\{l_{ik}^{(m-1)} + w_{kj}\right\}\right)$$

$$= \min_{1 \le k \le n}\left\{l_{ik}^{(m-1)} + w_{kj}\right\}$$

Shortest paths and matrix multiplication

A recursive solution to the all-pairs shortest-paths problem

- A path from vertex i to vertex j with more than $n-1$ edges cannot have lower weight than a shortest path from i to j.

$$\delta(i,j) = l_{ij}^{(n-1)} = l_{ij}^{(n)} = l_{ij}^{(n+1)} = \cdots.$$

Shortest paths and matrix multiplication

- We now compute a series of matrices $L^{(1)}, L^{(2)}, \ldots, L^{(n-1)}$, where for $m = 1,2, \ldots, n-1$, we have $L^{(m)} = (l_{ij}^{(m)})$.
- The procedure extends the shortest paths computed so far by one more edge.

Shortest paths and matrix multiplication

```
EXTEND-SHORTEST-PATHS(L, W)
1  n = L.rows
2  let L' = (l'_ij) be a new n × n matrix
3  for i = 1 to n
4      for j = 1 to n
5          l'_ij = ∞
6          for k = 1 to n
7              l'_ij = min(l'_ij, l_ik + w_kj)
8  return L'
```

Shortest paths and matrix multiplication

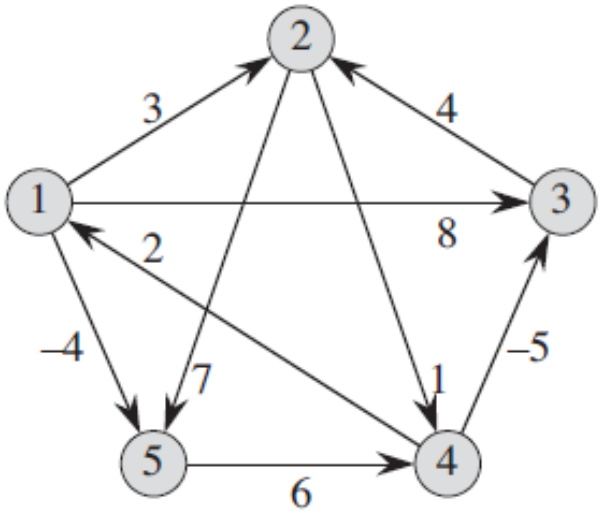
- It is similar to the procedure of matrix multiplication.

```
SQUARE-MATRIX-MULTIPLY(A, B)
1  n = A.rows
2  let C be a new n × n matrix
3  for i = 1 to n
4      for j = 1 to n
5          c_ij = 0
6          for k = 1 to n
7              c_ij = c_ij + a_ik · b_kj
8  return C
```

Shortest paths and matrix multiplication

```
SLOW-ALL-PAIRS-SHORTEST-PATHS(W)
1  n = W.rows
2  L^(1) = W
3  for m = 2 to n − 1
4       let L^(m) be a new n × n matrix
5       L^(m) = EXTEND-SHORTEST-PATHS(L^(m−1), W)
6  return L^(n−1)
```

Running time ?



$$L^{(1)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \qquad L^{(2)} = \begin{pmatrix} 0 & 3 & 8 & 2 & -4 \\ 3 & 0 & -4 & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & \infty & 1 & 6 & 0 \end{pmatrix}$$

$$L^{(3)} = \begin{pmatrix} 0 & 3 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} \qquad L^{(4)} = \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix}$$

Shortest paths and matrix multiplication

- The procedure can be improved by ***repeated squaring***.

$$
\begin{aligned}
L^{(1)} &= W\,, \\
L^{(2)} &= W^2 = W \cdot W\,, \\
L^{(4)} &= W^4 = W^2 \cdot W^2 \\
L^{(8)} &= W^8 = W^4 \cdot W^4\,, \\
&\vdots \\
L^{(2^{\lceil \lg(n-1) \rceil})} &= W^{2^{\lceil \lg(n-1) \rceil}} = W^{2^{\lceil \lg(n-1) \rceil - 1}} \cdot W^{2^{\lceil \lg(n-1) \rceil - 1}}\,.
\end{aligned}
$$

- Since $2^{\lceil \lg(n-1) \rceil} \geq n-1$, the final product $L^{(2^{\lceil \lg(n-1) \rceil})}$ is equal to $L^{(n-1)}$.

Shortest paths and matrix multiplication

```
FASTER-ALL-PAIRS-SHORTEST-PATHS(W)
1  n = W.rows
2  L^(1) = W
3  m = 1
4  while m < n − 1
5      let L^(2m) be a new n × n matrix
6      L^(2m) = EXTEND-SHORTEST-PATHS(L^(m), L^(m))
7      m = 2m
8  return L^(m)
```

Running time ?

Floyd-Warshall algorithm

The structure of a shortest path

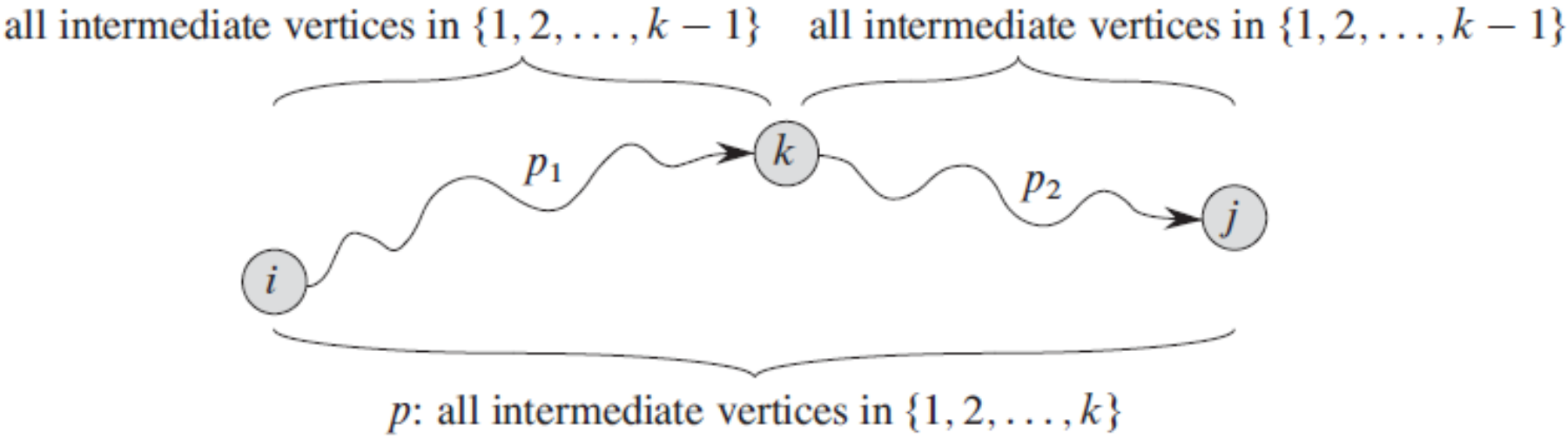
- An ***intermediate*** vertex of a simple path $p = \langle v_1, v_2, \ldots, v_l \rangle$ is any vertex of p other than v_1 or v_l, that is, any vertex in the set $\{v_2, v_3, \ldots, v_{l-1}\}$.
- For any pair of vertices $i, j \in V$, consider all paths from i to j whose intermediate vertices are all drawn from $\{1,2, \ldots, k\}$, and let p be a minimum-weight path from among them.

Floyd-Warshall algorithm

- The Floyd-Warshall algorithm exploits a relationship between path p and shortest paths from i to j with all intermediate vertices in the set $\{1,2,\ldots,k-1\}$. The relationship can be separated into two cases.
  - k is not an intermediate vertex of path p. A shortest path from vertex i to vertex j with all intermediate vertices in the set $\{1,2,\ldots,k-1\}$ is also a shortest path from i to j with all intermediate vertices in the set $\{1,2,\ldots,k\}$.

Floyd-Warshall algorithm

- k is an intermediate vertex of path p, then we decompose p into two paths.



Floyd-Warshall algorithm

A recursive solution to the all-pairs shortest-paths problem

- Let $d_{ij}^{(k)} = \begin{cases} w_{ij} & \text{if } k = 0 \\ \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}) & \text{if } k \geq 1 \end{cases}$
- The matrix $D^{(n)} = (d_{ij}^{(n)})$ gives the final answer

Floyd-Warshall algorithm

```
FLOYD-WARSHALL(W)
1  n = W.rows
2  D^(0) = W
3  for k = 1 to n
4      let D^(k) = (d_ij^(k)) be a new n × n matrix
5      for i = 1 to n
6          for j = 1 to n
7              d_ij^(k) = min(d_ij^(k-1), d_ik^(k-1) + d_kj^(k-1))
8  return D^(n)
```

Running time ?

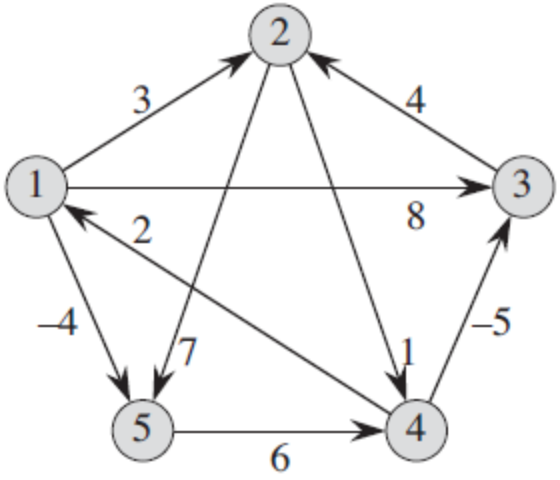
Floyd-Warshall algorithm

Constructing a shortest path

- A recursive formulation of $\pi_{ij}^{(k)}$

$$\pi_{ij}^{(0)} = \begin{cases} \text{NIL if } i = j \text{ or } w_{ij} = \infty \\ i \text{ if } i \neq j \text{ and } w_{ij} < \infty \end{cases}$$

$$\pi_{ij}^{(k)} = \begin{cases} \pi_{ij}^{(k-1)} \text{if } d_{ij}^{(k-1)} \leq d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \\ \pi_{kj}^{(k-1)} \text{if } d_{ij}^{(k-1)} > d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \end{cases}$$



$$D^{(0)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \qquad \Pi^{(0)} = \begin{pmatrix} \text{NIL} & 1 & 1 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & \text{NIL} & \text{NIL} \\ 4 & \text{NIL} & 4 & \text{NIL} & \text{NIL} \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(1)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \qquad \Pi^{(1)} = \begin{pmatrix} \text{NIL} & 1 & 1 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & \text{NIL} & \text{NIL} \\ 4 & 1 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(2)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \qquad \Pi^{(2)} = \begin{pmatrix} \text{NIL} & 1 & 1 & 2 & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & 2 & 2 \\ 4 & 1 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(3)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \qquad \Pi^{(3)} = \begin{pmatrix} \text{NIL} & 1 & 1 & 2 & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & 2 & 2 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(4)} = \begin{pmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} \qquad \Pi^{(4)} = \begin{pmatrix} \text{NIL} & 1 & 4 & 2 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(5)} = \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} \qquad \Pi^{(5)} = \begin{pmatrix} \text{NIL} & 3 & 4 & 5 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix}$$

Acknowledgements & References

- T.H. Cormen, C.E. Leiserson, R.L. Rivest, and C. Stein, Introduction to Algorithms, 3rd edition, The MIT Press, 2009.